

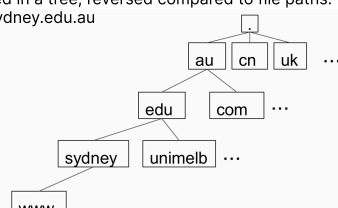
Detection: Large data streams are broken into packets. Problems: packets may be lost, corrupted, or arrive out of order. **IP checksum** detects most corruption. **CRC (Cyclic Redundancy Check)** is more powerful; computed using modulo-2 arithmetic (**XOR**). **Used in Ethernet frames**. **TCP** reliable, ordered, error-checked: Sequence numbers allow reordering. Lost packets are retransmitted. **ACK** confirms delivery. Used for applications requiring reliability (web, email, SSH). **UDP** fast, no guarantee: Fire-and-forget datagrams. No ordering, no retransmission. Used for **real-time apps (voice, gaming)**. **DNS**, streaming. **ICMP Internet Control Message Protocol:** is used for sending control messages such as echo requests and time-exceeded messages. **ping** & **traceroute**: Ping tests if a host is reachable by sending **ICMP echo requests**, while Traceroute finds the path to a destination by sending packets with increasing **TTL values**; each router returns a "TTL exceeded" message, revealing the route hop by hop. **ip a (ifconfig -a)/ip** shows network config, route -n or ip route shows routing table.

traceroute -n -q 1 -w 2 www.google.com
-n: Do not perform reverse DNS lookups. This makes the output faster (it shows raw IPs directly).
-w 2: Set the per-hop wait time to 2 seconds.
-q 1: Send only 1 probe per hop (default is 3).
Two major classes of **routing**: LAN, WAN. Routing determines the path IP packets take from the source to the destination, forwarding them **hop-by-hop** using routing tables. **LANs (Ethernet/WiFi)** use **broadcast shared medium**. Devices only process frames with their own MAC address, or broadcast MAC (FF:FF:FF:FF:FF:FF). **ARP Address Resolution Protocol:** To discover the ethernet (MAC) address of a device given its IP address. **ARP** message: "Who has 129.78.8.X?" Owner replies with its **MAC address**. **Sender** caches result. **LAN Address Blocks:** A LAN is assigned a **contiguous block** (e.g., 129.78.8.0/24 = 256 IPs). To check if an IP is inside the LAN: Apply **subnet mask** (e.g., 255.255.255.0). **If masked result equals the LAN base, it's local.** If destination is NOT inside the LAN: Send packet to the gateway router ("default gateway"). Gateway forwards the packet to the wider Internet.

Each host/router uses a **routing table** like: (using 'route')
Destination Gateway Genmask Iface
default gw 0.0.0.0 em1
129.78.8.0 * 255.255.252.0 em1
Rules: If destination matches a specific network entry → use that entry. Otherwise → use default route. The global Internet = many interconnected networks called **Autonomous Systems (AS)**. Routers exchange routes using **BGP (Border Gateway Protocol)**. AS numbers now 32 bits due to growth (>60,000 ASes). Solution to shortage of address: **private networks + NAT**.
Private IPv4 Ranges (IANA-reserved): 10.0.0.0 to 10.255.255.255
172.16.0.0 to 172.31.255.255, 192.168.0.0 to 192.168.255.255
Most home WiFi uses 192.168.x.x.

NAT Network Address Translation Inside LAN: devices use private IPs. **Router** translates private IP + port into a single public IP + port. **Allows thousands of private hosts to share one public address**.
192.168.1.100:1234 → router → 120.23.45.1:24789 (public)
NAT works by replacing the source IP and port of outgoing packets with the router's public IP and a unique port, keeping a translation table so it can forward the return traffic to the correct internal device.
IPv6 Long-Term Solution: 128-bit (8*4*4) addresses. Enough addresses for the entire planet many times over. Better routing efficiency. Adoption slow because IPv4 + NAT works well.
UDP/TCP and Applications: Some specifics: DNS as noted uses UDP (and sometimes TCP for large responses like zone transfers). (DNS lookup uses UDP with IP of course as underlying network layer). HTTP uses TCP (usually port 80). SMTP email uses TCP (port 25). SSH uses TCP (22). Streaming protocols or real-time like VoIP often run over UDP (with additional protocols to handle loss).

DNS and application layer protocols: Domain Name System (DNS): solves the problem of mapping human-readable names to IP addresses. **A Distributed, Hierarchical Namespace**. Domain names are arranged in a tree, reversed compared to file paths.
www.sydney.edu.au



Nameservers: Each level has authoritative nameservers that know the records below them. Your computer uses a preconfigured DNS resolver, usually provided via DHCP.

DNS Lookup Name Process: a recursive resolver (often provided by the ISP or local DNS resolver (from DHCP)). Resolver will check its cache; if not found: Resolver asks **root server** to get the address of **TLD server** (.com, .au, etc.) → asks TLD server to get the **authoritative server** for the domain → asks authoritative server to get final IP. The resolution uses UDP on **port 53** for queries/replies in most cases. **DNS protocol sends messages using UDP**. If UDP responses are too large it may use TCP. Each DNS entry is a **Resource Record: A → IPv4; AAAA → IPv6; MX → Mail server; CNAME → Alias; MX (mail exchanger for domain); TXT (text info); NS (nameserver for the domain); PTR (reverse lookup pointer)**. **DHCP (Dynamic Host Configuration Protocol):** When a device joins the network, it **broadcasts** a request. DHCP provides: Your IP address; Default gateway; DNS server. **To assign IP config automatically**. Example: an A record: www.sydney.edu.au → 129.78.xxx.xxx. MX request say sydney.edu.au → mail.sydney.edu.au. The DNS query can request a specific record type. The lecture likely mentioned some, e.g., A, MX, NS. Caching and TTL: DNS replies include a Time-to-Live. Resolvers cache records to reduce load. That's why changes to DNS may take time to propagate (you wait for caches to expire).

Application Layer (User Data) Protocols:
Ports (TCP/UDP): Ports **identify applications** within a host. 16-bit

Port Number	Protocol
20, 21	File Transfer Protocol (FTP)
22	Secure Shell (SSH)
23	Telnet Protocol
25	Simple Mail Transfer Protocol (SMTP)
53	Domain Name System (DNS)
67, 68	Dynamic Host Configuration Protocol (DHCP)
80	HyperText Transfer Protocol (HTTP)
110	Post Office Protocol (POP3)
137	NetBIOS Name Service
143	Internet Message Access Protocol (IMAP4)
443	Secure HTTP (HTTPS)
445	Microsoft-DS (Active Directory)

HTTP (HyperText Transfer Protocol): Used to transfer web pages. Runs over TCP port 80.
Client:
GET /index.html HTTP/1.1
Server:
HTTP/1.1 200 OK
Content-Type: text/html; charset=UTF-8
<html>
<head>
<title>An Example Page</title>
</head>
<body>
Hello World, this is a very simple HTML document.
</body>
</html>

SMTP (Simple Mail Transfer Protocol): Used to send email. Process: Client finds destination domain's MX record via DNS; Client opens TCP connection to mail server (port 25); Exchanges text-based commands (HELO, MAIL FROM, RCPT TO, DATA, QUIT)
Email Format (MIME Standard): Emails are formatted as: Header (From, To, Subject, etc.); Blank line; Body. MIME allows: Attachments; Embedded images; Non-text content
Summary Example: Visiting http://www.sydney.edu.au/
Your PC checks cache or asks DNS for www.sydney.edu.au → gets an IP. It opens a TCP connection to that IP on port 80. Sends an HTTP GET request for "/" page. Server responds with HTML content. Closes connection (or reuses for more requests). Under the hood, many protocols worked: ARP (to reach default gateway), IP routing, TCP reliability, DNS resolution, etc., but the user just sees the webpage load. The nc command makes a simple, unencrypted, connection to a remote host and port that you specify.

Cloud computing : **Cloud computing:** A network of remote servers hosted on the Internet to store, manage, and process data, rather than a local server or a personal computer. Key features: **on-demand self-service**, broad network access, resource pooling (multi-tenancy), (scale up/down quickly), and measured service (pay-as-you-go).
Cloud Service Models: IaaS (Infrastructure as a Service): Provides virtual machines, storage, networking. **User manages OS + applications**. Example: **Amazon EC2 (Elastic Compute Cloud):** user selects CPU, RAM, region; cloud provisions VMs on physical servers. **PaaS (Platform as a Service):** Provides runtime environment + databases without exposing OS or hardware. **Developer deploys apps directly**; provider handles scaling. Example: Google App Engine, Azure App Service. **SaaS (Software as a Service):** Full applications delivered as a service. User only uses the software; **no infrastructure management**. Example: Gmail, Office 365, Salesforce.

Major Cloud Providers: **AWS (largest)**, Microsoft Azure, Google Cloud, IBM Cloud. Originated as internal large-scale computing systems before being offered externally.

How Cloud Works (IaaS): Cloud providers run large data centers filled with physical servers (**hypervisors**). A user requests a VM with specific specs (CPU, RAM, storage, region). The cloud orchestration system finds available capacity and launches the VM. Multiple VMs may share one physical server, isolated by the hypervisor. Billing is typically per second or per hour, plus extra for storage and bandwidth. Users interact with VMs remotely (SSH, browser, etc.). Cloud APIs allow automated provisioning (e.g., create more VMs when load is high). A running VM is called an **"instance"** and can be stopped and restarted easily.
Scaling & APIs: Cloud platforms expose APIs (Application Program Interface) to programmatically create, destroy, or resize resources. Example: auto-scaling — when CPU usage > 80%, automatically launch a new instance. When load decreases, instances terminate to save cost. To scale a web service: use auto-scaling groups + load balancer + CDN + managed database. **Key Cloud Services (AWS Examples):** **Compute:** EC2 (VMs), Lambda (serverless functions), ECS/EKS (containers); **Storage:** S3 (Simple Storage Service) (object storage: data, metadata, key), EBS (block storage), Glacier (archival storage; cheaper but slow retrieval); **Databases:** RDS (relational), DynamoDB (NoSQL); **Networking:** VPC, Route53 (DNS), API Gateway, load balancers; **Security:** IAM, KMS, monitoring, DDoS protection.
Data Centres: AWS operates dozens of regions and availability zones worldwide. Distributed for: Reduced latency, **Redundancy**, Cheaper and greener energy sources
Serverless computing: **AWS Lambda.** You deploy a function; the cloud executes it on demand. No servers to manage; **scales automatically**; billed per request/runtime. No idle cost.

IoT & Edge Computing: **IoT (Internet of Things):** Cloud handles processing for low-power devices sending telemetry. **Edge computing:** Moves computation closer to users/devices for low latency, lower bandwidth use, higher reliability. Often implemented with CDNs (Amazon CloudFront).
Microservices Architecture: Cloud systems often use microservices: **small, independent** services deployed in containers or as serverless functions. They communicate over **APIs or message queues**. **Benefits:** independent scaling, modularity, parallel development.
Economic and Practical Impact: Avoids large upfront hardware costs; Pay only for what you use; Easy global deployment; Encourages designing for failure (instances may be killed; redundancy required) Traditional: fixed servers running 24/7, limited scalability. Cloud: dynamic capacity — launch 1 server or 100 servers instantly and shut them down anytime.
Challenges: Security misconfigurations (open S3 buckets); Vendor lock-in; Requires reliable internet; Must manage permissions and access carefully. Cloud billing models include On-Demand (pay per second/hour), Reserved Instances (cheaper long-term commitment), and Spot Instances (lowest cost but interruptible).

Computer and network security: Security Goals: Network security aims to guarantee four core objectives: **Confidentiality** (Prevent unauthorized access; data stays secret (e.g., protect passwords)). **Integrity** (Ensure data is not modified undetectably). **Authenticity** (Know who you are communicating with (identity verification)), non-repudiation (Sender cannot deny having sent a message (digital signatures)).
Cryptography Basics: **Plaintext (m):** original message. **Ciphertext (E(m))**: scrambled message. **Key (k):** required to encrypt/decrypt. Encryption/decryption are inverses: **D(E(m,k),k) = m** (eg. Caesar cipher) Goal: attackers may see ciphertext but must not recover plaintext. **Symmetric (Secret-Key) Cryptography:** **Same key** used for encryption and decryption. Very fast, ideal for bulk encryption (HTTPS data, storage). Require secure key distribution between parties. Exam tip: RSA is not a symmetric algorithm even though it has a "private key". **Asymmetric (Public-Key) Cryptography:** Uses a pair: **Public key:** shared openly, **Private key (k_priv):** kept secret. Alice → Bob Encryption: Alice encrypts with Bob's public key. Only Bob can decrypt using his private key. This eliminates the shared-secret problem. The public key is shared openly and used to encrypt messages, while the private key is kept secret and used to decrypt messages. **Important Algorithms:** **RSA:** based on factoring; supports both encryption and signatures; **Diffie-Hellman:** key exchange protocol, not for encryption; enables two parties to agree on a shared secret over an insecure channel; **ECC:** elliptic curves; small keys, strong security. Public-key crypto is much **slower**, so only used for key setup.
Hybrid Encryption (Used in HTTPS & SSH): uses public key cryptography to exchange a symmetric key, then uses the symmetric key for fast encryption. Both sides now use fast symmetric encryption (AES). This is how real secure communication works (TLS handshake).
Digital Signatures (Authenticity + Non-repudiation): Sender generates a hash of the message → Sender encrypts the hash with their private key to create a digital signature → Sender sends the message and signature to the receiver → Receiver generates a hash of the received message → Receiver decrypts the signature with the sender's public key → Receiver compares both hashes to verify authenticity. Goal: prove who sent the message and guarantee it wasn't altered. **Using RSA-style systems:** Signing: Alice signs message (or its hash) using her private key. Verification: Anyone verifies using Alice's public key. Only Alice possesses the private key → authenticity + non-repudiation.

Signature workflow: Compute hash H(m); Sign H(m) with private key; Receiver verifies: public-key decrypt(signature) == H(m)
Algorithms: RSA signatures, DSA, ECDSA.
Hash Functions (Integrity): Hash = fixed-length output representing data. Good hash properties: One-way; Collision-resistant; Deterministic. Examples: SHA-256, SHA-384 (outputs 384 bits = 96 hex digits), SHA-512. Used for: Integrity checks; Password storage (store hash, not plaintext); Digital signatures (sign the hash, not the whole message).
TLS / HTTPS: **HTTPS = HTTP over TLS** → provides encryption + integrity + authentication. **TLS process:** Browser connects to server → Server sends certificate (signed by Certificate Authority) → Browser verifies the certificate with trusted CA public keys → Browser and server perform key exchange (RSA or Diffie-Hellman) → Both switch to fast symmetric encryption (AES). This prevents man-in-the-middle attacks and guarantees authenticity (you are talking to the real server).
Certificates & Certificate Authorities (CAs): A certificate is: "CA X vouches that public key Y belongs to server Z". Browser trusts certain CA public keys. Fake websites cannot present a valid CA-signed certificate. Essential for verifying server identity in HTTPS.
SSH & SCP: SSH uses **asymmetric keys** for authentication (public on server, private on client). After handshake, a symmetric session key encrypts traffic. SCP uses SSH to securely transfer files.
Common Algorithms Summary: Symmetric: AES, DES (legacy), RC4 (legacy). Asymmetric: RSA, Diffie-Hellman (key exchange), ECC/ECDHE, ECDSA. Hash: SHA-256, SHA-384, SHA-512

Performance Analysis: Understand where time goes and why systems behave slowly. Why: In distributed systems and modern applications, performance bottlenecks can degrade user experience and increase costs. Small delays across thousands of microservice calls add up. Also, inefficient resource usage can mean needing more servers (higher cost). So we analyze performance to optimize **reliability** and **scalability**. **Performance Metrics:** **Latency:** time per request. **Throughput:** Requests processed per second (RPS). Throughput and latency interact: high load usually increases latency. **Utilisation:** CPU/memory/network used. **Scalability:** performance as we add nodes. **Queueing delay:** when demand > service rate.
Total Packet Delay: $d_{total} = d_{proc} + d_{queue} + d_{trans} + d_{prop}$
dproc (Processing Delay): = processing delay, time to examine packet header and determine output link
dqueue (Queueing Delay): = queueing delay, time packet waits in buffer before transmission
dtrans (Transmission Delay): L/R (packet length / bandwidth), time to push all packets onto link
dprop (Propagation Delay): d/s (link distance / propagation speed), time for a bit to travel across link
Caravan Analogy: Toll booth = transmission delay. Cars = bits, caravan = packet. Time = push caravan through + propagation.
Queueing Theory Simple Model (M/M/1): 1. Arrivals are random 2. One server, so link transmits one packet at a time 3. Traffic intensity for a single server or link: $p = \lambda a / R$ = $\lambda \times \text{packet size} \times \text{arrival rate (packets/s)} / R$ = transmission rate. **Light load (p ≈ 0) minimal queueing, p = 0.5 - 0.7 (moderate) p > 0.9 (unstable).** This explains why networks collapse suddenly under heavy load.

Real Internet Measurement: **traceroute:** sends 3 packets per hop → measures round-trip delay. No response = probe lost, router not replying shown with * symbol). Trans-oceanic links show large jumps. Packet Loss Happens when queue is full → packets dropped. May be retransmitted or lost permanently.
Throughput Rate: rate (bits/time unit) at which bits transferred between sender/receiver. Instantaneous = rate at given point in time. RS is the server's **sending rate**; R server bits/sec, RC is the client's **receiving rate**, and the **bottleneck** link is the **slowest** link on the path, which determines the maximum possible throughput between them.
Performance Measurement: Time on system B / time on system A = A is times as fast as B. $((\text{time on B} / \text{time on A}) - 1) \times 100\% = \text{System A is } x\% \text{ faster than System B}$. **Arithmetic Mean** = average, **Weighted Average** = sum of execution x frequency. Use when workloads have known frequencies. **Geometric Mean:** A relative to B: $x_i = TA_i / TB$, $GM = (x_1 \times x_2 \times \dots \times x_n)^{1/n}$ Good for comparing **relative performance**. Downsides: not proportional to real speed. For geometric mean A relative to B, take ratios $A/B = x$, then apply formula **Harmonic Mean:** $n / (1/x_1 + 1/x_2 + \dots + 1/x_n)$ Best when **dealing with rates**.
Pitfalls in Measurement: Selective statistics (only best results reported). Reporting peak instead of average. Vague comparisons ("nearly", "more"). Wrong metrics (apples vs. oranges). Implying "Everyone buys this system".
Benchmarking: Used to compare performance objectively. No single benchmark is universally representative. CPU frequency alone is misleading. Real performance depends on: cores/threads/cache/memory hierarchy/microarchitecture + ILP power/thermal limits/compiler + workload. **Modern Benchmark Categories:** SPEC CPU – CPU + memory. CloudSuite – cloud workloads. TPC-C / TPC-H – database/transaction processing. MLPerf – AI/ML training + inference.